CESI
ÉCOLE D'INGÉNIEURS

Stockage & Big Data

Workshop | Manipulation et analyse de données avec Hive

Mercredi 28 octobre 2026









Mazou Marine
Fisa Info 25–28
CESI

TABLE DES MATIÈRES

Introduction **2**

Étude des données **3**

Partie 1 : base de données et tables **4**
1. Préparer HDFS et créer la base 4
2. Table externe et suppression 5
3. Table finale par CTAS 5
4. Table interne et suppression 6

Partie 2 : partitions et buckets **7**
1. Partition par sexe 7
2. Partitions et buckets sur les internautes 8
3. Buckets sur le pays, sans partition 10

Partie 3 : analyse des salaires **11**
1. Préparation 11
2. Q1 à Q7 13
3. Q8 à Q10 et conclusion 15

Pièges des données et écarts à l'énoncé **19**
1. Pièges des données 19
2. Écarts à l'énoncé 19

Conclusion **20**

INTRODUCTION

Apache Hive est un entrepôt de données bâti sur Hadoop. Il permet d'interroger des fichiers stockés dans HDFS avec le langage **HiveQL**, très proche du SQL : Hive traduit chaque requête en travaux MapReduce. Hive sépare les données (des fichiers dans HDFS) de leur description (le *metastore*, qui contient les noms de tables, les colonnes et les emplacements). Une table peut donc être de deux types :

- **externe** (`EXTERNAL`) : Hive ne décrit que des fichiers qui existent déjà. Supprimer la table ne supprime pas les fichiers ;
- **interne** (gérée) : Hive possède les fichiers. Supprimer la table les supprime aussi.

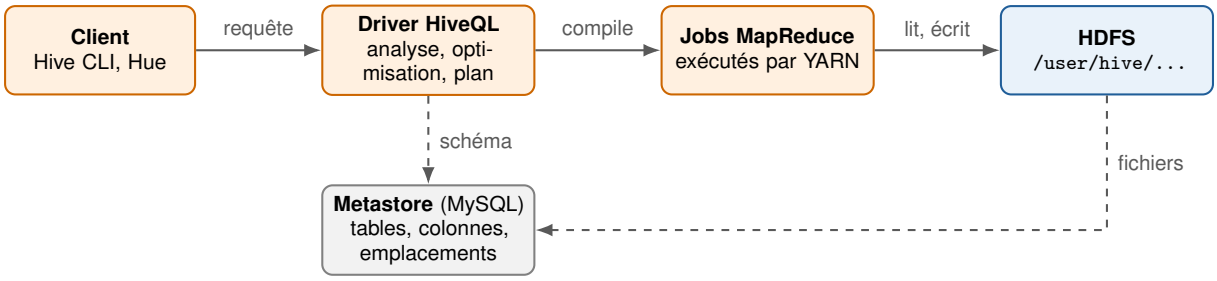


Figure 1 – Architecture de Hive sur la machine Cloudera : le metastore décrit les tables, HDFS contient les fichiers.

Le compte-rendu commence par une étude des six fichiers de données, puis le workshop comporte trois parties :

1. créer une base et des tables, externes puis internes, et observer ce que devient le fichier à la suppression ;
2. organiser les données avec des **partitions** et des **buckets** ;
3. analyser les salaires d'une entreprise avec dix requêtes HiveQL.

Tout est exécuté sur la machine virtuelle **Cloudera quickstart 5.12** (Hive 1.1.0-cdh5.12.0), avec un script par partie. Le script `00_hdfs.sh` prépare les dossiers HDFS, puis `01_base_et_tables.hql`, `02_partitions_buckets.hql` et `03_analyse.hql` se lancent avec `hive -f`. Chaque fichier de données est placé dans son propre dossier HDFS : une table externe lit tous les fichiers de son dossier.

ÉTUDE DES DONNÉES

Avant d'écrire une table, les six fichiers fournis ont été analysés : leur forme décide du type des colonnes et des options de lecture. Le tableau donne leur contenu, puis les particularités à connaître.

Fichier	Lignes	Contenu	Utilisé en
`emploi_population.txt`	12	Genre, date de naissance, fonction, salaire, code postal, id	Partie 1
`personne.txt`	12	Id, nom, prénoms	Partie 1 (table interne)
`internautes_log.txt`	17	Id, prénom, site consulté, date, pays	Partie 2
`employes.txt`	33	Id, nom, prénoms, date de naissance	Partie 3
`fonction_employes.txt`	33	Id, année d'embauche, fonction	Partie 3
`salaire_employes.txt`	33	Sexe, secteur, salaire, code postal, id	Partie 3

Les nombres de lignes comptent les données, sans l'en-tête (`fonction_employes.txt` et `salaire_employes.txt` ont 34 lignes avec lui). Les volumes qui comptent pour la suite :

- **33 employés**, répartis en **16 secteurs** (partie 3) ;
- **17 lignes de journal**, sur **3 dates** (27, 28 et 29 avril 2016) et **6 pays** (partie 2) ;
- **12 personnes** dans les deux fichiers de la partie 1.

Les fichiers ont en commun leur séparateur, le point-virgule, écrit `';'` (son code octal) dans les scripts. Les autres caractéristiques diffèrent d'un fichier à l'autre :

Caractéristique	Constat	Conséquence pour Hive
Encodage	`fonction_employes.txt` et `salaire_employes.txt` étaient en ISO-8859-1 ; les quatre autres sont en ASCII	Accents illisibles (« profession juridique privé », « année-embauche ») tant qu'ils ne sont pas en UTF-8
Fins de ligne	Les six fichiers d'origine étaient en CRLF	Hive garde le `\r` dans la dernière colonne, ce qui donne `NULL` sur un entier
En-têtes	Une ligne d'en-tête dans `fonction_employes.txt` et `salaire_employes.txt` seulement	Cette ligne serait lue comme une donnée
Dates	`emploi_population.txt` : AAAA-MM-JJ ; `employes.txt` : JJ/MM/AAAA ; `internautes_log.txt` : JJ-MM-AAAA	Seul le premier format est accepté par le type `DATE`
Salaire	Une chaîne du type `20000euro/an`, avec une faute à la ligne 26 (`42000euo/an`)	Pas de calcul possible sans extraire les chiffres

Le traitement appliqué à chacun de ces points est détaillé dans la section « Pièges des données et écarts à l'énoncé ».

PARTIE 1 : BASE DE DONNÉES ET TABLES

1. PRÉPARER HDFS ET CRÉER LA BASE

Le script `00_hdfs.sh` donne les droits sur l'entrepôt Hive, crée `/user/hive/data` et y copie chaque fichier dans un sous-dossier propre. La base `htables` est ensuite créée avec un emplacement choisi, puis une propriété lui est ajoutée :

```
CREATE DATABASE IF NOT EXISTS htables
COMMENT 'Base de données du WS Hive'
LOCATION '/user/hive/data/htables';
USE htables;
ALTER DATABASE htables SET DBPROPERTIES ('owner'='A3', 'date'='2022-02-03');
DESCRIBE DATABASE EXTENDED htables;
```

La sortie réelle de `DESCRIBE DATABASE EXTENDED` confirme l'emplacement et les propriétés :

Champ	Valeur lue sur Hive
location	hdfs://quickstart.cloudera:8020/user/hive/data/htables
owner_name	cloudera (USER)
parameters	{owner=A3, date=2022-02-03}

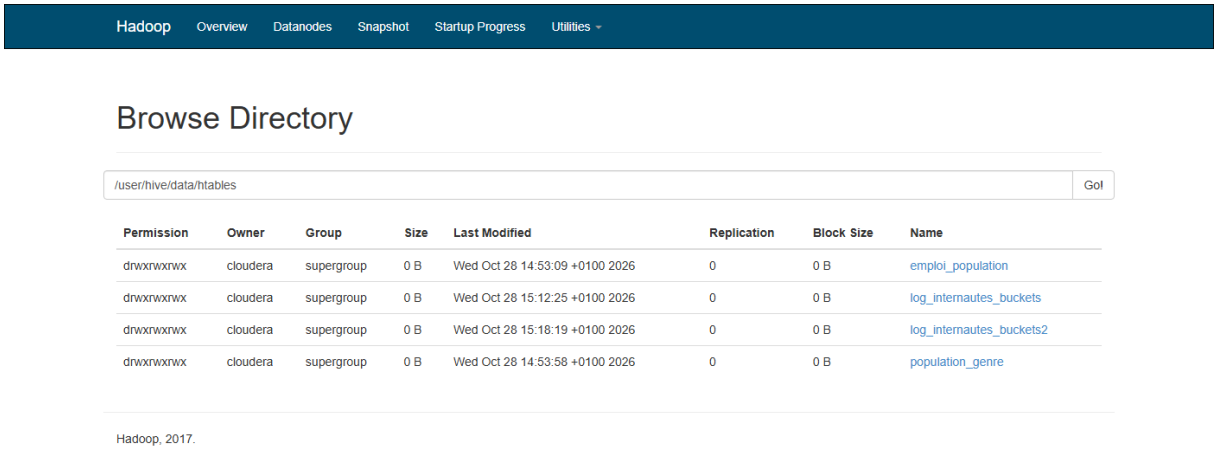


Figure 2 – Navigateur HDFS : dossier `/user/hive/data/htables` après la création de la base.

2. TABLE EXTERNE ET SUPPRESSION

La table externe `log_emploi_population` décrit le fichier `emploi_population.txt`, sans le déplacer :

```
CREATE EXTERNAL TABLE IF NOT EXISTS log_emploi_population (
  genre STRING, date_naissance DATE, fonction STRING,
  salaire STRING, code_postal STRING, id INT)
COMMENT 'table intermediaire'
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
STORED AS TEXTFILE
LOCATION '/user/hive/data/emploi_population';
```

`DESCRIBE FORMATTED` indique `Table Type: EXTERNAL_TABLE` et `field.delim = ;`. Le `SELECT *` renvoie bien les 12 lignes (extrait réel) :

genre	date_naissance	fonction	salaire	code_postal	id
Feminin	1989-03-03	Aide-soignant	20000euro/an	59000	1
Masculin	1982-02-15	Informaticien	35000euro/an	80000	2
Feminin	1968-04-26	Infermiere	22000euro/an	77000	3
		...			
Feminin	1996-03-20	Musicienne	41000euro/an	6000	12

Que devient le fichier si on supprime la table ? Le script exécute `DROP TABLE log_emploi_population`, puis `dfs -ls /user/hive/data/emploi_population`. Le fichier `emploi_population.txt` (634 octets) est toujours là : Hive n'a supprimé que la description. La table recréée retrouve ses données immédiatement.

3. TABLE FINALE PAR CTAS

La table `emploi_population` est créée à partir de la table intermédiaire avec un *CREATE TABLE AS SELECT* : le genre est réduit à son initiale (F ou M) et la date de naissance devient un âge.

```
CREATE TABLE IF NOT EXISTS emploi_population AS
SELECT SUBSTR(genre, 1, 1) AS sexe,
       ROUND(DATEDIFF(CURRENT_TIMESTAMP(), date_naissance) / 365.2425) AS age,
       fonction, salaire, code_postal, id AS Id_empl
FROM log_emploi_population;
```

La requête garde le `CURRENT_TIMESTAMP()` de l'énoncé, qui fonctionne dans le Hive 1.1 de Cloudera. Le `DESCRIBE` réel donne les colonnes `sexe string`, `age double`, `fonction`, `salaire`, `code_postal` (`string`) et `id_empl int`. L'âge est un `double` parce que `ROUND` renvoie un `double`. Extrait de la table, pour les femmes (sortie réelle de `SELECT * ... WHERE sexe = 'F'`, âges au 28 octobre 2026) :

sexe	age	fonction	salaire	code_postal	id_empl
F	38,0	Aide-soignant	20000euro/an	59000	1
F	59,0	Infermiere	22000euro/an	77000	3
F	70,0	Professeur	30000euro/an	95000	4
F	71,0	Pericultrice	38000euro/an	76000	8
F	54,0	Informaticienne	45000euro/an	91000	9
F	31,0	Musicienne	41000euro/an	6000	12

Le flux complet de la partie 1, du fichier à la table finale :

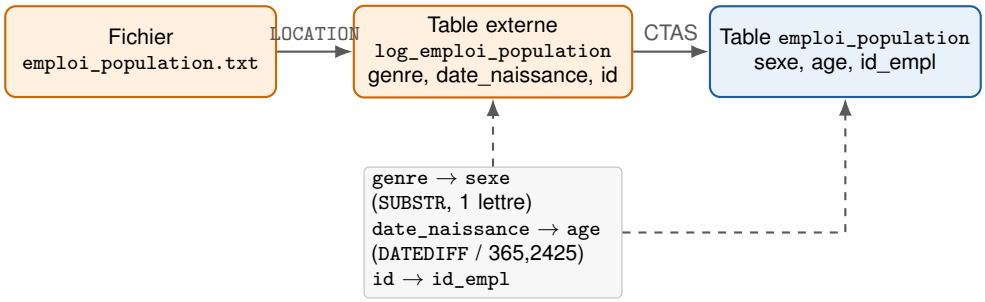


Figure 3 – Passage de la table intermédiaire à la table finale : colonnes transformées par le CTAS.

4. TABLE INTERNE ET SUPPRESSION

La table interne `personne` reçoit le fichier `personne.txt`, copié au préalable dans l'entrepôt Hive. L'énoncé laisse sa définition à compléter : nous retenons deux colonnes, `id INT` et `nom STRING` (le fichier a une troisième colonne, les prénoms, ignorée par la table).

```
CREATE TABLE IF NOT EXISTS personne (id INT, nom STRING)
COMMENT 'Table interne personne'
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE;
LOAD DATA INPATH '/user/hive/warehouse/personne.txt' OVERWRITE INTO TABLE
↪  personne;
DROP TABLE personne;
dfs -ls /user/hive/data/htables;
```

`LOAD DATA INPATH` **déplace** le fichier : il quitte `/user/hive/warehouse` pour `/user/hive/data/htables/personne`. Après le `DROP TABLE`, le `dfs -ls` de `/user/hive/data/htables` ne montre plus que le dossier `emploi_population` : contrairement à la table externe, la table interne emporte ses données.

	Table externe	Table interne
Propriétaire des fichiers	Les fichiers existent avant la table	Hive (entrepôt ou emplacement de la base)
`DROP TABLE`	Supprime la description, garde les fichiers	Supprime la description et les fichiers
`LOAD DATA INPATH`	Non utilisé (`LOCATION`)	Déplace le fichier vers la table
Exemple du workshop	`log_emploi_population`	`personne`

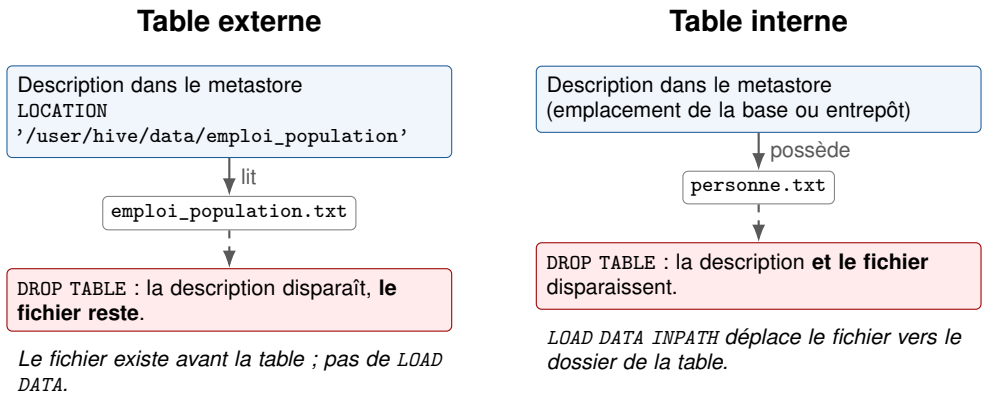


Figure 4 – Table externe et table interne : où sont les données et effet de `DROP TABLE`.

PARTIE 2 : PARTITIONS ET BUCKETS

---

1. PARTITION PAR SEXE

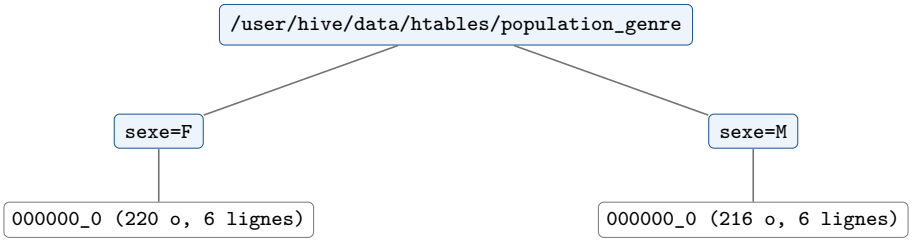
Une **partition** range les lignes dans un sous-dossier par valeur d'une colonne. Une requête qui filtre sur cette colonne ne lit que le sous-dossier concerné. La table `population_genre` est partitionnée par `sexe` ; la colonne n'est plus dans les fichiers, elle est dans le nom du dossier.

```
CREATE EXTERNAL TABLE population_genre (
  age INT, fonction STRING, salaire STRING, code_postal STRING, id INT)
PARTITIONED BY (sexe STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';

SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
INSERT OVERWRITE TABLE population_genre PARTITION (sexe)
SELECT age, fonction, salaire, code_postal, id_empl, sexe FROM emploi_population;
```

Le mode dynamique crée les partitions à partir des valeurs lues ; le mode `nonstrict` est nécessaire parce que toutes les colonnes de partition sont dynamiques. Résultats réels :

- `SELECT DISTINCT(sexe)` renvoie `F` et `M` ;
- `SHOW PARTITIONS population_genre` renvoie `sexe=F` et `sexe=M` ;
- le journal de l'insertion donne 6 lignes dans chaque partition (`numRows=6`, `totalSize` de 220 et 216 octets) ;
- `dfs -ls -R` montre un fichier `000000_0` dans chacun des deux sous-dossiers.



Les deux requêtes suivantes donnent les mêmes six femmes, mais la seconde ne lit que le dossier `sexe=F` :

```
SELECT * FROM emploi_population WHERE sexe = 'F';
SELECT * FROM population_genre  WHERE sexe = 'F';
```

Table	Temps	Mode d'exécution lu dans le journal
`emploi_population` (complète)	13,949 s	Travail MapReduce, 1 mapper, aucun reducer
`population_genre (partition F)`	0,21 s	Aucun travail MapReduce : lecture directe du dossier

Les deux temps sont lus dans la sortie réelle du rejeu final. Ils ne se comparent pas directement : la requête sur la table complète a lancé un travail MapReduce (environ 14 s, dont l'essentiel est le démarrage du travail), alors que celle sur la partition a été lue sans MapReduce, par un simple parcours du dossier (0,21 s). Sur douze lignes, le gain vient de la *méthode* et non du volume : la partition évite de lancer un travail et de lire les fichiers de l'autre sexe. Ce gain grandit avec le volume des données.

2. PARTITIONS ET BUCKETS SUR LES INTERNAUTES

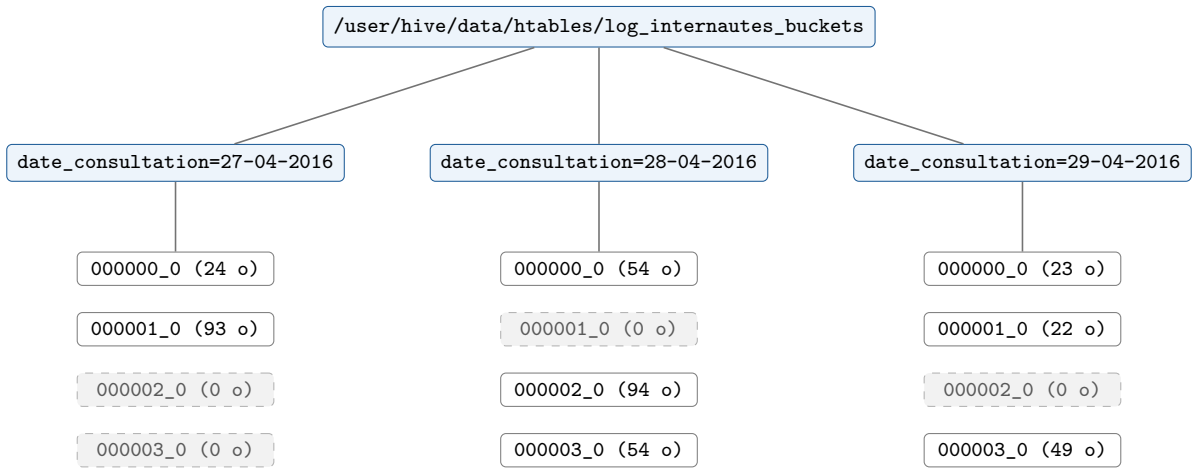
Un **bucket** répartit les lignes d'une table (ou d'une partition) dans un nombre fixe de fichiers, selon le hachage d'une colonne : bucket = (hachage & `Integer.MAX_VALUE`) modulo 4. Pour un entier, le hachage est sa valeur ; pour une chaîne, c'est `String.hashCode()`. Les buckets servent à répartir la charge, à échantillonner (`TABLESAMPLE`) et à accélérer les jointures.

La table intermédiaire `log_internautes` (17 lignes) est lue depuis le dossier `/user/hive/data/internautes`. Une première table réunit les deux techniques : une partition par date, et dans chaque partition 4 buckets sur `id_internaute`.

```
SET hive.enforce.bucketing=true;
CREATE TABLE log_internautes_buckets (
  id_internaute INT, prenom STRING, site_consulte STRING, pays STRING)
PARTITIONED BY (date_consultation STRING)
CLUSTERED BY (id_internaute) INTO 4 BUCKETS
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE;

INSERT OVERWRITE TABLE log_internautes_buckets PARTITION (date_consultation)
SELECT id_internaute, prenom, site_consulte, pays, date_consultation FROM
↪  log_internautes;
```

`DESCRIBE FORMATTED` confirme `Num Buckets: 4` et `Bucket Columns: [id_internaute]` ; `SHOW PARTITIONS` liste les trois dates. Le travail MapReduce utilise 4 reducers, un par bucket. L'arborescence réelle relevée par `dfs -ls -R` est la suivante (taille de chaque fichier en octets) :



Les fichiers en pointillés sont des buckets vides (0 octet) : ils existent quand même, parce que Hive crée toujours autant de fichiers que de buckets. Le bucket de chaque ligne se calcule avec la règle ci-dessus ; les tailles lues correspondent au calcul (la répartition par bucket est calculée avec cette règle, le total de chaque partition est lu dans `numRows`) :

Partition	Bucket 0	Bucket 1	Bucket 2	Bucket 3	Total (lu)
27-04-2016	1	4	0	0	5
28-04-2016	2	0	4	2	8
29-04-2016	1	1	0	2	4

Pour le 27 avril, les buckets 2 et 3 sont vides : avec cinq lignes et quatre buckets, un bucket sans ligne est normal. Cela confirme la remarque de l'énoncé. Les deux autres dates ont aussi un bucket vide.

Les captures du navigateur HDFS confirment ces fichiers : les trois dossiers de partition (Figure 5), puis le contenu de la partition du 27 avril, où `000002_0` et `000003_0` (les buckets 2 et 3) font bien 0 octet (Figure 6).

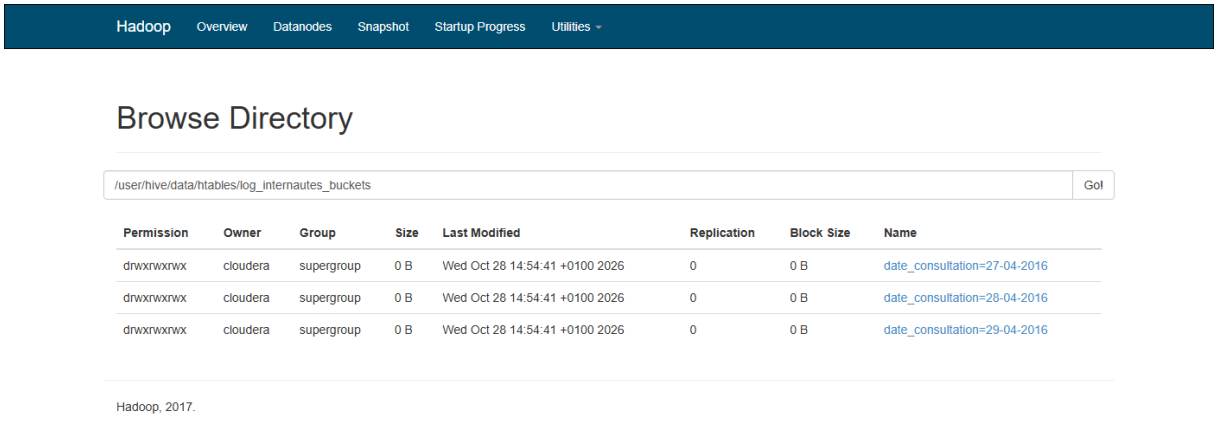


Figure 5 – Navigateur HDFS : les trois partitions de `log_internautes_buckets`.

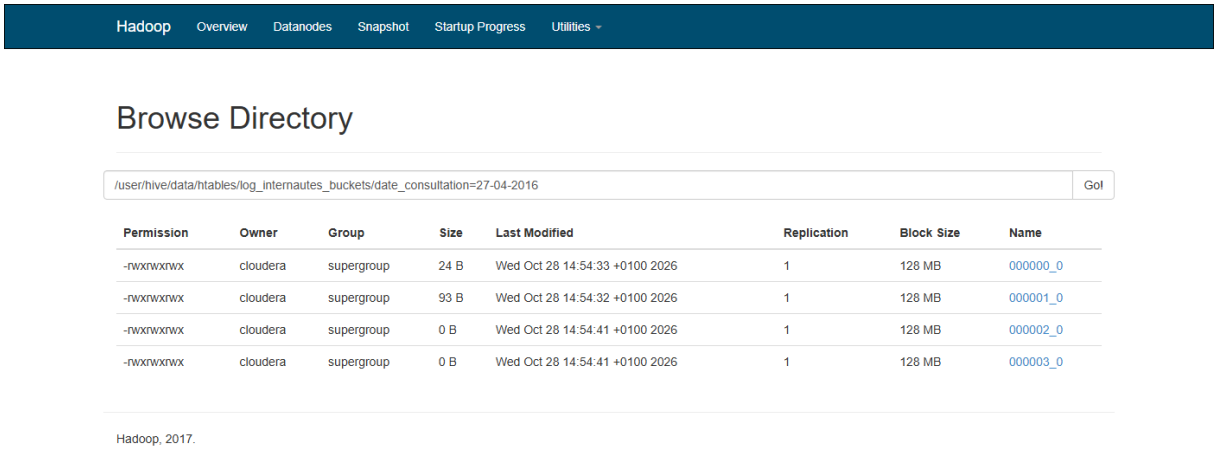


Figure 6 – Partition du 27-04-2016 : `000000_0` (24 o), `000001_0` (93 o), `000002_0` et `000003_0` (0 o).

3. BUCKETS SUR LE PAYS, SANS PARTITION

La seconde table répartit toutes les lignes en 4 buckets selon `pays`, sans partition :

```
CREATE TABLE log_internautes_buckets2 (
  id_internaute INT, prenom STRING, site_consulte STRING,
  date_consultation STRING, pays STRING)
CLUSTERED BY (pays) INTO 4 BUCKETS
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE;
```

Le journal donne `numFiles=4, numRows=17, totalSize=600`. Les fichiers lus par `dfs -ls -R` :

Fichier	Taille (o)	Pays qu'il contient
000000_0	84	AUSTRALIA (2 lignes, vérifié par `TABLESAMPLE`)
000001_0	0	Aucun
000002_0	305	JAPAN, UK, US (9 lignes, calculé par le hachage)
000003_0	211	CHINA, INDIA (6 lignes, calculé par le hachage)

Six pays se partagent quatre buckets : le hachage ne garantit pas une répartition égale. Le bucket 1 reste vide, et le bucket 2 regroupe trois pays. Le choix de la colonne de bucket compte : avec peu de valeurs distinctes, la répartition est inégale. La requête `TABLESAMPLE(BUCKET 1 OUT OF 4 ON pays)` ne lit que le premier fichier ; elle renvoie bien les deux lignes d'Australie (Ricky et Michel).

PARTIE 3 : ANALYSE DES SALAIRES

1. PRÉPARATION

La base `analyse_salaires` contient trois tables externes, une par fichier. Les fichiers `fonction_employes` et `salaire_employes` ont une ligne d'en-tête, ignorée grâce à `skip.header.line.count` :

```
CREATE EXTERNAL TABLE salaire_employes (
  sexe STRING, secteur STRING, salaire STRING, code_postal INT, id_empl INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/hive/data/salaire'
TBLPROPERTIES ("skip.header.line.count"="1");
```

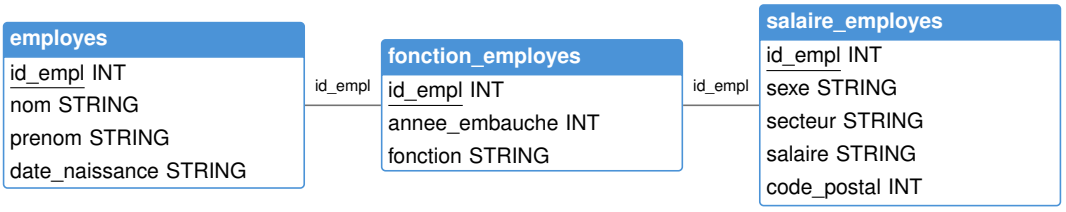


Figure 7 – Les trois tables externes de `analyse_salaires`, reliées par la clé `id_empl` (33 lignes chacune).

Les trois `SELECT COUNT(*)` renvoient bien **33** lignes chacune (`employes`, `fonction_employes`, `salaire_employes`), lu sur Hive. Le salaire est une chaîne du type `20000euro/an`. Une vue utilitaire, ajoutée hors énoncé, joint les trois tables et convertit le salaire en nombre ; l'expérience est l'année courante moins l'année d'embauche.

```sql
CREATE VIEW v_employes_complet AS
SELECT e.id_empl, e.nom, e.prenom, s.sexe, s.secteur,
       CAST(REGEXP_REPLACE(s.salaire, '[^0-9]', '') AS INT) AS salaire_num,
       s.code_postal, f.annee_embauche, f.fonction,
       YEAR(CURRENT_DATE()) - f.annee_embauche AS experience
FROM employes e
JOIN fonction_employes f ON e.id_empl = f.id_empl
JOIN salaire_employes s ON e.id_empl = s.id_empl;
```

La regex supprime tout ce qui n'est pas un chiffre : elle traite aussi la faute `42000euo/an` de la ligne 26. Sur la machine Cloudera, les jointures échouent avec la conversion automatique en jointure locale (`MapredLocalTask`, erreur Java) : le script ajoute `SET hive.auto.convert.join=false`. Les résultats ci-dessous sont les résultats réels de Hive, lus dans la sortie du rejeu complet du script ; chaque requête MapReduce prend de 20 s à 2 ou 3 minutes sur la machine virtuelle.

L'interface YARN (`quickstart.cloudera:8088`) liste les travaux MapReduce lancés par les requêtes : les 62 travaux de la session ont le statut *SUCCEEDED*.

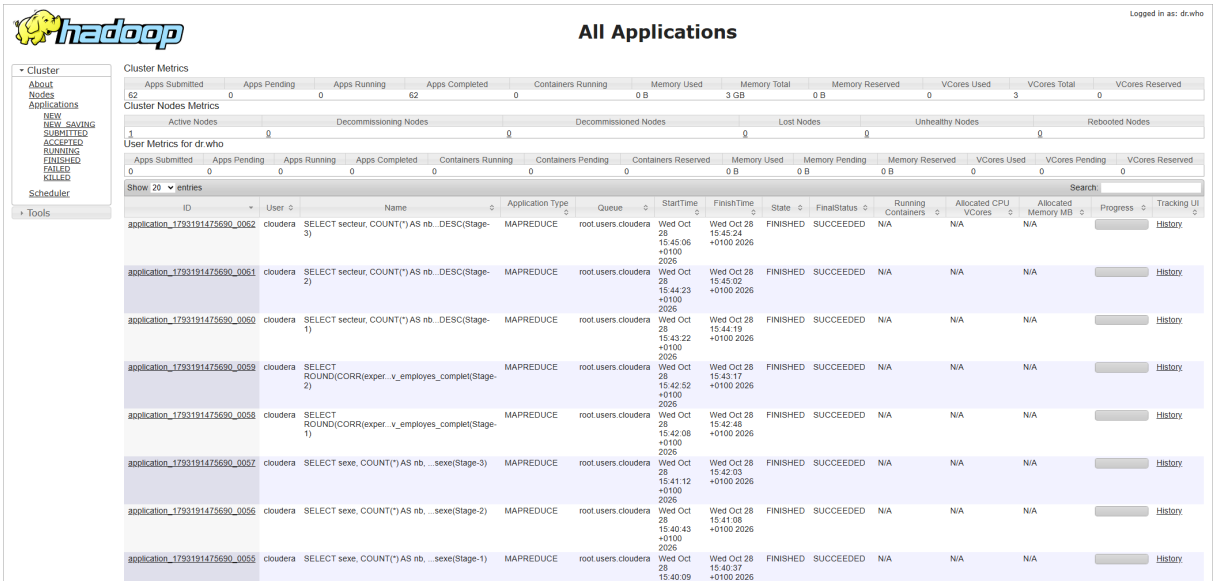


Figure 8 – YARN, All Applications : travaux MapReduce lancés par les requêtes Hive (62 travaux SUCCEEDED).

2. Q1 À Q7

Q1. Combien d'employés ont été embauchés après 2019 ?

```
SELECT COUNT(*) AS nb_embauches_apres_2019
FROM fonction_employes WHERE annee_embauche > 2019;
```

Résultat lu sur Hive : **7** employés. L'année 2019 est exclue, d'après le sens de « après ».

Q2. Quelles paires employé-secteur ont un salaire supérieur à 35 000 euros par an ?

```
SELECT id_empl, nom, prenom, secteur, salaire_num
FROM v_employes_complet WHERE salaire_num > 35000 ORDER BY id_empl;
```

24 lignes (les salaires de 35 000 exactement, ids 2 et 27, sont exclus). Les huit premières :

id	nom	prenom	secteur	salaire
5	VANDENDRIESSCHE	HENRI PIERRE	industrie	45 000
6	WACCUS	GUILLAUME AUGUSTIN	profession juridique privé	70 000
7	CHARPENTIER	PIERRE	sante	45 000
8	DEKYDTSPOTTER	PAULETTE OCTAVIE	soin et education	38 000
9	SOLEYMIEUX	FRANCINE JEANNE	industrie	45 000
11	TERZIS	PASCAL PIERRE	sante	55 000
12	HOUMADI OILI	FATIMA	arts et spectacle	41 000
13	LALLEMANT	ETIENNE BERNARD PIERRE	transports aeriens	144 000
...				

La plus haute valeur est 200 000 (id 30, profession juridique privé).

Q3. Quels employés du secteur éducation ont été recrutés après 2010 ? L'énoncé crée une vue avec `employes` et `fonction_employes` ; le secteur est dans `salaire_employes`, d'où une jointure de plus.

```
CREATE VIEW vue_employes_apres_2010 AS
SELECT e.id_empl, e.nom, e.prenom, f.annee_embauche, f.fonction
FROM employes e JOIN fonction_employes f ON e.id_empl = f.id_empl
WHERE f.annee_embauche > 2010;

SELECT v.id_empl, v.nom, v.prenom, v.annee_embauche, v.fonction
FROM vue_employes_apres_2010 v JOIN salaire_employes s ON v.id_empl = s.id_empl
WHERE s.secteur = 'education' ORDER BY v.id_empl;
```

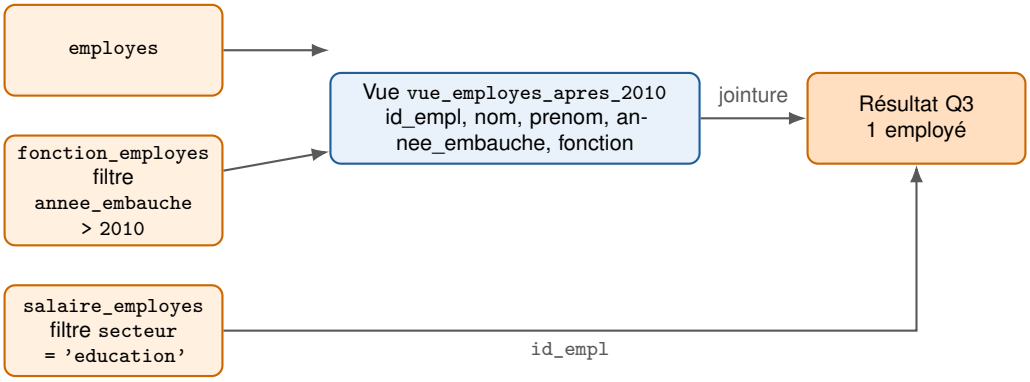


Figure 9 – Question Q3 : la vue de l'énoncé (employés recrutés après 2010) est jointe à `salaire_employes` pour filtrer le secteur.

Une seule personne : **CANSALADE Charlotte Henriette** (id 19, embauchée en 2018, fonction « ressource humaine »). Le filtre est une égalité stricte : le secteur « soin et education » n'est pas retenu.

Q4. Quel employé a le plus d'expérience ? L'expérience est l'année courante moins l'année d'embauche. La requête garde les ex-aequo avec `RANK()` ; Hive 1.1 n'accepte pas de sous-requête scalaire.

```
SELECT id_empl, nom, prenom, annee_embauche, fonction, experience
FROM (SELECT id_empl, nom, prenom, annee_embauche, fonction, experience,
             RANK() OVER (ORDER BY experience DESC) AS rang
      FROM v_employes_complet) t
WHERE rang = 1;
```

TODORIS Vassiliki (id 4, professeur, embauchée en 1970), soit 56 ans d'expérience en 2026, sans ex-aequo.

Q5. Quel est le salaire moyen des employés ?

```
SELECT ROUND(AVG(salaire_num), 2) AS salaire_moyen FROM v_employes_complet;
```

55 696,97 euros par an (1 838 000 pour 33 employés).

Q6. Quelles sont les fonctions des femmes, par année d'embauche ? `COLLECT_SET` regroupe les fonctions distinctes de chaque année dans une liste.

```
SELECT annee_embauche, COLLECT_SET(fonction) AS fonctions
FROM v_employes_complet WHERE sexe = 'F'
GROUP BY annee_embauche ORDER BY annee_embauche;
```

Année	Fonctions	Année	Fonctions
1970	professeur	2011	chercheur en chimie
1977	pericultrice	2015	aide-soignant
1984	transporteur	2018	musicienne, ressource humaine
1987	infermiere, secretaire, professeur	2020	infermiere, comptable
1990	pediatre	2021	phamacienne
2010	informaticienne	2022	technicien

Les listes sont données dans l'ordre renvoyé par Hive ; cet ordre n'est pas garanti.

Q7. Quelles paires <prénom, fonction> concernent les employés qui habitent Paris ? Le code postal doit commencer par 75.

```
SELECT prenom, fonction, code_postal
FROM v_employes_complet WHERE code_postal BETWEEN 75000 AND 75999
ORDER BY code_postal;
```

prenom	fonction	code_postal
DOUGLAS LOUIS LEON	taxi	75000
PIERRE	veterinaire	75780

La colonne `code_postal` est ajoutée au `SELECT` : Hive 1.1 refuse un `ORDER BY` sur une colonne absente du `SELECT`. Le code 75780 n'est pas un code postal parisien, mais il passe le filtre 75xxx. Le prénom contient tous les prénoms de l'état civil.

3. Q8 À Q10 ET CONCLUSION

Q8. Y a-t-il une différence de salaire entre les femmes et les hommes ? Le salaire seul ne tient pas compte de l'ancienneté. Le ratio retenu est la somme des salaires divisée par la somme des années d'expérience, par sexe.

```
SELECT sexe, COUNT(*) AS nb, SUM(salaire_num) AS somme_salaires,
       SUM(experience) AS somme_experience,
       ROUND(SUM(salaire_num) / SUM(experience), 2) AS
       ↪  ratio_salaire_par_annee_exp,
       ROUND(AVG(salaire_num), 2) AS salaire_moyen
FROM v_employes_complet GROUP BY sexe ORDER BY sexe;
```

sexe	nb	Somme salaires	Somme exp.	Ratio	Salaire moyen
F	16	644 000	379	1 699,21	40 250,00
M	17	1 194 000	332	3 596,39	70 235,29

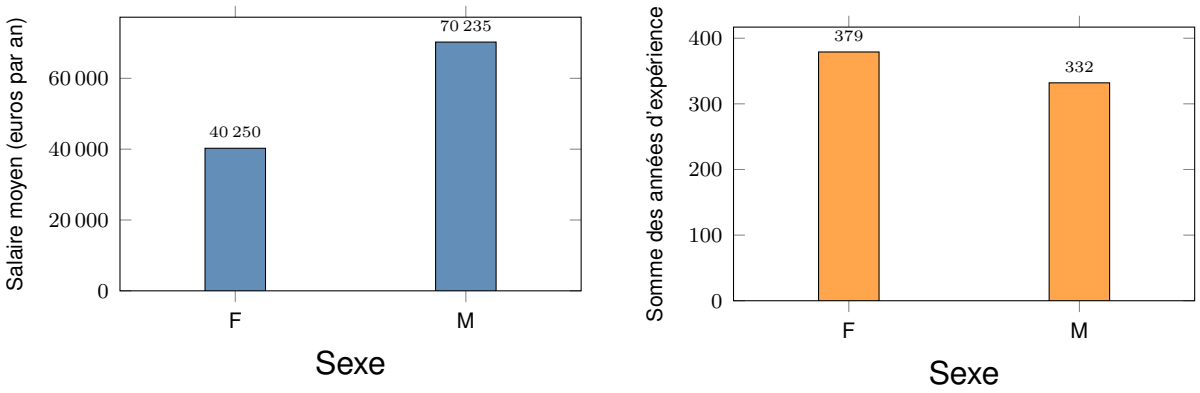


Figure 10 – Q8 : salaire moyen et somme des années d'expérience, par sexe (16 femmes, 17 hommes).

Le salaire moyen des hommes est supérieur de près de 30 000 euros à celui des femmes. Le ratio des hommes est 2,12 fois celui des femmes, alors que les femmes cumulent plus d'années d'expérience (379 contre 332). L'écart n'est donc pas expliqué par l'ancienneté : il se creuse quand on la prend en compte. L'échantillon est de 33 personnes et sans test statistique : c'est un constat, pas une preuve.

Q9. L'expérience influence-t-elle le salaire ? La fonction `CORR` donne le coefficient de corrélation de Pearson.

```
SELECT ROUND(CORR(experience, salaire_num), 4) AS correlation_experience_salaire
FROM v_employes_complet;
```

Résultat : **0,0886**. La corrélation est positive mais très faible (proche de 0) : sur ces données, l'expérience n'explique presque pas le salaire.

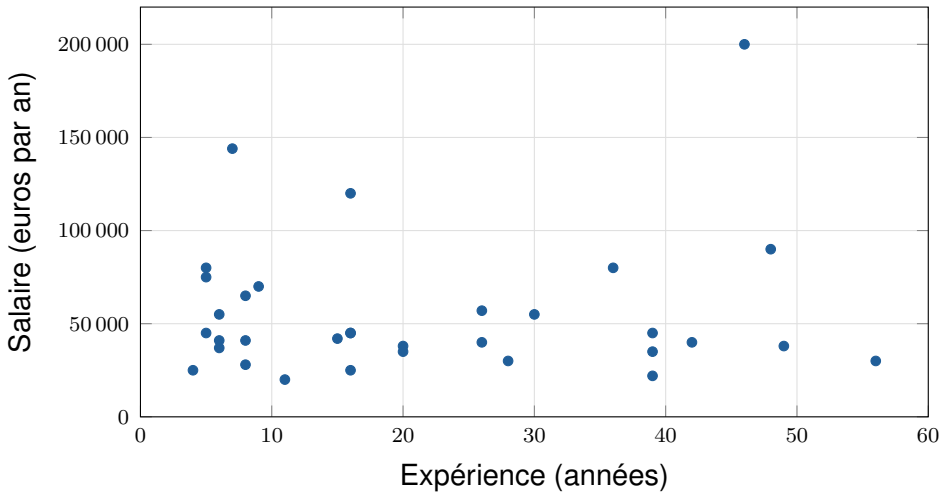


Figure 11 – Q9 : salaire en fonction de l'expérience pour les 33 employés (corrélation de Pearson : 0,0886). Le nuage est dispersé, sans tendance nette.

Q10. Le secteur influence-t-il le salaire ?

```sql
SELECT secteur, COUNT(*) AS nb, ROUND(AVG(salaire_num), 2) AS salaire_moyen,
       ROUND(AVG(experience), 2) AS experience_moyenne
FROM v_employes_complet GROUP BY secteur ORDER BY salaire_moyen DESC;
```

Secteur	nb	Salaire moyen	Exp. moyenne
profession juridique privé	2	135 000,00	27,50
transports aeriens	2	132 000,00	11,50
transport	2	85 000,00	26,50
fabrication et distribution des patisserie-viennoiserie	1	65 000,00	8,00
pharmaceutique	2	58 500,00	10,00
agriculture	2	43 500,00	27,00
sante	7	42 571,43	19,14
education	4	39 500,00	33,25
industrie	4	35 000,00	10,25
… (16 secteurs au total)			

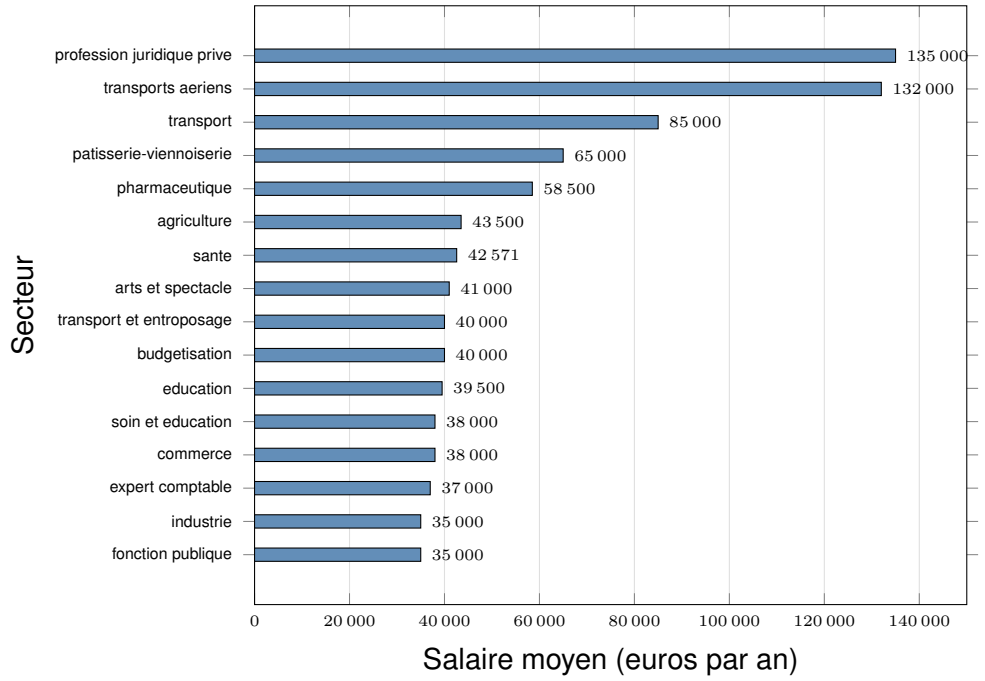
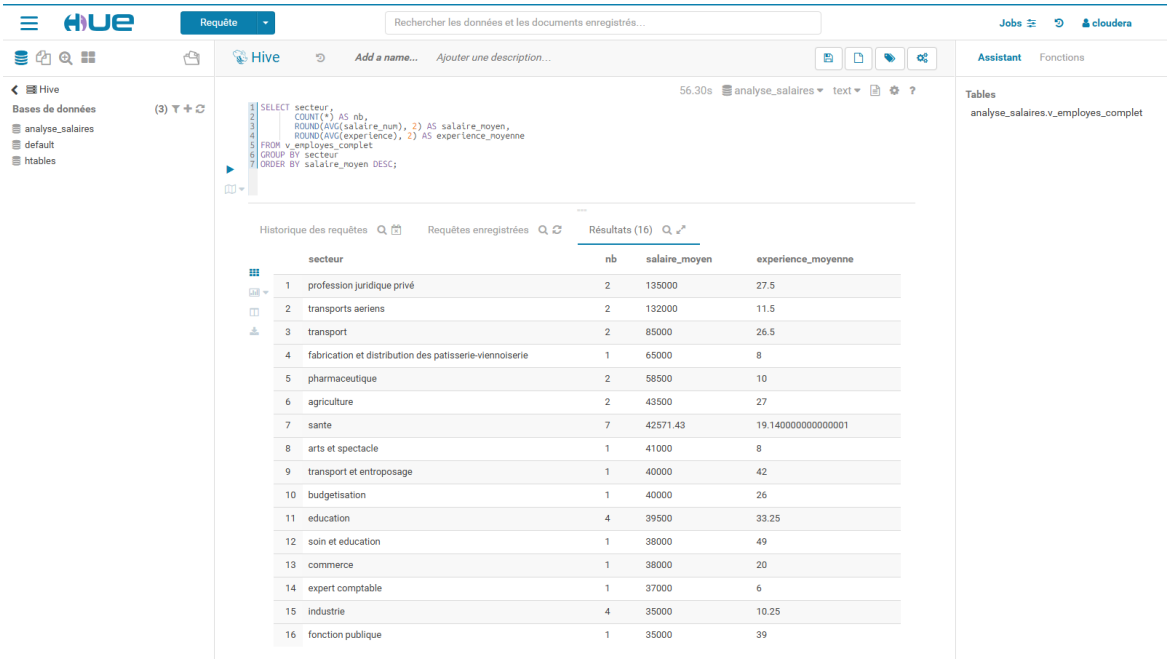


Figure 12 – Q10 : salaire moyen par secteur, du plus élevé (en haut) au plus bas (16 secteurs).

La capture Hue montre la requête et ses 16 lignes de résultat (56,30 s). La valeur `19.140000000000001` de l'expérience moyenne du secteur « sante » est un artefact d'affichage du type `double` : il s'agit bien de 19,14.



	secteur	nb	salaire_moyen	experience_moyenne
1	profession juridique privé	2	135000	27.5
2	transports aeriens	2	132000	11.5
3	transport	2	85000	26.5
4	fabrication et distribution des patisserie-viennoiserie	1	65000	8
5	pharmaceutique	2	58500	10
6	agriculture	2	43500	27
7	sante	7	42571.43	19.140000000000001
8	arts et spectacle	1	41000	8
9	transport et entroposage	1	40000	42
10	budgetisation	1	40000	26
11	education	4	39500	33.25
12	soin et education	1	38000	49
13	commerce	1	38000	20
14	expert comptable	1	37000	6
15	industrie	4	35000	10.25
16	fonction publique	1	35000	39

Figure 13 – Hue, éditeur Hive : requête Q10 et résultat (16 secteurs).

Les moyennes vont de 35 000 à 135 000 euros. Les secteurs les mieux payés (juridique, transports aériens, transport) n'ont pas les employés les plus expérimentés : le transport aérien a 11,5 ans d'expérience en moyenne. **Conclusion Q8 à Q10** : le secteur influence nettement le salaire, alors que l'expérience seule n'a pas d'effet net (Q9). Le sexe est lié à un écart important (Q8), mais il peut aussi venir du secteur : ces données ne permettent pas de séparer les deux effets. Réserve : neuf secteurs sur seize n'ont qu'un ou deux employés, et les valeurs de Q4, Q8, Q9 et Q10 dépendent de l'année d'exécution.

PIÈGES DES DONNÉES ET ÉCARTS À L'ÉNONCÉ

1. PIÈGES DES DONNÉES

Les cinq premiers pièges sont décrits dans la section « Étude des données » ; le tableau donne le traitement de chacun, puis ceux qui concernent l'énoncé et les requêtes.

Piège	Traitement appliqué
Encodage	Conversion en UTF-8 (copies dans `data/`)
Fins de ligne	Conversion en LF (`dos2unix`) avant la copie sur la VM
En-têtes	`skip.header.line.count=1` sur `fonction_employes` et `salaire_employes`
Dates	Colonnes en `DATE` pour AAAA-MM-JJ ; `employes.txt` en `STRING`, inutilisé dans les requêtes
Salaire en texte	`CAST(REGEXP_REPLACE(salaire, '[^0-9]', '') AS INT)`, qui corrige aussi `euo/an`
Noms de fichiers	L'énoncé cite des fichiers `.csv` (`employes1.csv`...) ; les fichiers sont des `.txt`, noms réels utilisés
Colonnes en trop	`personne.txt` a trois colonnes, l'énoncé n'en demande que deux : le troisième champ est ignoré
Prénoms	La colonne `prenom` de `employes` contient tous les prénoms de l'état civil : à signaler (Q7)
Code 75780	Passe le filtre 75xxx de Q7 sans être parisien : conservé, signalé

2. ÉCARTS À L'ÉNONCÉ

- `CURRENT_TIMESTAMP()` : la fonction existe dans le Hive 1.1 de Cloudera ; la syntaxe de l'énoncé est conservée dans le CTAS. Les questions utilisent `YEAR(CURRENT_DATE())` pour l'année courante.
- **Emplacement de la table externe** : `/user/hive/data/emploi_population` au lieu de `/user/hive/data`. Ce dernier dossier contient des sous-dossiers que Hive ne sait pas lire sur une table texte (erreur *Not a file*).
- **Nom de table** : l'énoncé demande de supprimer `log_salaire_population`, qui n'existe pas ; la table s'appelle `log_emploi_population`.
- **Base `htables`** : l'énoncé la crée deux fois (`CREATE DATABASE`, puis `CREATE DATABASE IF NOT EXISTS ... LOCATION`, sans effet sur une base existante). Le script ne garde que la seconde, précédée d'un `DROP DATABASE IF EXISTS ... CASCADE` pour pouvoir être rejoué.
- **Rejeu de la partie 1** : `LOAD DATA` déplace `personne.txt` hors de l'entrepôt ; il faut relancer `00_hdfs.sh` avant de rejouer le script.
- **Table `personne`** : définition laissée libre par l'énoncé, choisie à deux colonnes (`id`, `nom`).

- `population_genre` : table externe sans `LOCATION`, comme dans l'énoncé ; ses partitions sont écrites dans le dossier de la base `htables`. La colonne `age INT` reçoit un `double` : conversion implicite.
- **Q3** : la vue de l'énoncé ne contient pas le secteur, d'où une jointure avec `salaire_employes`.
- **Q4** : `RANK()` plutôt qu'une sous-requête scalaire, non acceptée par Hive 1.1.
- **Séparateur `';'`** : le client Hive coupe l'instruction au point-virgule placé dans la chaîne (erreur *ParseException ... expecting StringLiteral near 'BY'*). Le script écrit `'\073'`, le code octal du point-virgule.
- **Jointures** : `SET hive.auto.convert.join=false` en tête du script 03, parce que la conversion en jointure locale échoue sur la machine (`MapredLocalTask`, *NullPointerException*).
- **Q7** : `ORDER BY id_empl` échoue (*Invalid table alias or column reference*) car la colonne n'est pas dans le `SELECT` ; la requête sélectionne `code_postal` et trie dessus.
- **Ajouts** : la vue `v_employes_complet`, des commandes `dfs -ls` dans les scripts pour voir les fichiers sans l'interface web, et `SET hive.enforce.bucketing=true` (valable en Hive 1.1, supprimé en Hive 4).
- **Dépendance à la date** : Q4, Q8, Q9, Q10 et l'âge de `emploi_population` changent avec la date d'exécution (ici, l'année 2026 et le 28 octobre 2026).

CONCLUSION

Ce workshop a présenté trois outils de Hive pour manipuler des données dans HDFS :

1. **les tables externes et internes** : la table externe décrit des fichiers qui lui survivent, la table interne possède les siens et les supprime avec elle ;
2. **les partitions et les buckets** : la partition range les lignes par valeur d'une colonne, ce qui évite de lire les autres dossiers ; le bucket répartit les lignes en un nombre fixe de fichiers par hachage, avec des fichiers vides et des répartitions inégales quand il y a peu de lignes ou de valeurs distinctes ;
3. **l'analyse en HiveQL** : jointures, agrégats, fonctions de fenêtrage (`RANK`), listes (`COLLECT_SET`) et corrélation (`CORR`) sur trois tables d'employés.

L'analyse montre que le secteur pèse nettement sur le salaire, et que l'expérience seule n'a presque pas d'effet. Un écart entre les femmes et les hommes apparaît, sans que l'on puisse le séparer de l'effet du secteur sur 33 personnes. Hive 1.1 impose ses limites (fonctions absentes, sous-requêtes, jointures locales) : plusieurs requêtes de l'énoncé ont dû être adaptées, et chaque écart est noté. Enfin, les données ont demandé un nettoyage avant toute requête (encodage, fins de ligne, en-têtes), comme dans le workshop précédent.